Homework # 2 – Solutions

1. Is it possible to design a recursive descent parser for the below grammar? Explain why or why not?

S → AbB | bAc

A → Ab | aBB

B → Ac | cBb | c

Answer: To design a recursive descent parser, grammar must satisfy the below two properties at the same time for all rules in the given grammar:

i. No indirect or no direct left recursion occurs in the grammar.

S → AbB | bAc   // No left recursion

A → Ab | aBB   // Direct left recursion

B → Ac | cBb | c   // No left recursion

As one of the rules has direct left recursion, it is not possible to write a recursive descent parser.

ii. Rules must pass the pairwise disjointness test

S → AbB | bAc   // {b} ∩ {a} = ∅   this rule passes the pairwise disjointness test

A → Ab | aBB   // {a} and no other terminal symbol, this rule passes the pairwise disjointness test.

B → Ac | cBb | c   // {a, c} ∩ {c} ≠ ∅   this rule cannot pass the pairwise disjointness test. Therefore it is not possible to write a recursive descent parser for the given grammar.

2. Eliminate the direct left-recursion for the following grammar rules.

A → Aa | Ab | aaB | cBB

B → BAa | BabA | aBb | cB

C → CBc | aC | CBac

Answer:

The first grammar rule should be replaced with the below two rules:

A → aaBA' | cBBA'

A' → aA' | bA' | Ɛ

The second grammar rule should be replaced with the below two rules:

B → aBbB' | cBB'

B' → AaB' | abAB' | Ɛ

The third grammar rule should be replaced with the below two rules:

C → aCC'

C → BcC' | BacC' | Ɛ

3. Show a complete parse, including the parse stack contents, input string, and action for the string (id + id) * (id + id) $, according to the shift-reduce algorithm using the grammar and parse table given in class.

Answer:

Stack	**Input String**	**Action**
0	**(** id + id) * (id + id) $	S4
0 (**4**	**id** + id) * (id + id) $	S5
0 (4 id **5**	**+** id) * (id + id) $	R6
0 (4 F **3**	**+** id) * (id + id) $	R4
0 (4 T **2**	**+** id) * (id + id) $	R2
0 (4 E **8**	**+** id) * (id + id) $	S6
0 (4 E 8 + **6**	**id**) * (id + id) $	S5
0 (4 E 8 + 6 id **5**	**)** * (id + id) $	R6
0 (4 E 8 + 6 F **3**	**)** * (id + id) $	R4
0 (4 E 8 + 6 T **9**	**)** * (id + id) $	R1
0 (4 E **8**	**)** * (id + id) $	S11
0 (4 E 8) **11**	***** (id + id) $	R5
0 F **3**	***** (id + id) $	R4
0 T **2**	***** (id + id) $	S7
0 T 2 * **7**	**(** id + id) $	S4
0 T 2 * 7 (**4**	**id** + id) $	S5
0 T 2 * 7 (4 id **5**	**+** id) $	R6
0 T 2 * 7 (4 F **3**	**+** id) $	R4
0 T 2 * 7 (4 T **2**	**+** id) $	R2
0 T 2 * 7 (4 E **8**	**+** id) $	S6
0 T 2 * 7 (4 E 8 + 6	**id**) $	S5
0 T 2 * 7 (4 E 8 + 6 id **5**	**)** $	R6
0 T 2 * 7 (4 E 8 + 6 F **3**	**)** $	R6
0 T 2 * 7 (4 E 8 + 6 T **9**	**)** $	R1
0 T 2 * 7 (4 E **8**	**)** $	S11
0 T 2 * 7 (4 E 8) **11**	**$**	R5
0 T 2 * 7 F **10**	**$**	R3
0 T **2**	**$**	R2
0 E **1**	**$**	**accept**

The string is syntactically correct.